

Project Report EE524 (Broadwell)

Mohammad Chahardori, Kevin Teo, Mohammad Ali Mokri

Abstract—In this report, we present the effectiveness of three different methods of branch prediction in two different programs. The three methods used were Local, Global, and Tournament Prediction algorithms, which were implemented in MATLAB. To verify the impact of the three prediction schemes, we use two different benchmarks. The first benchmark is Espresso, which is a standard example of an integer program, and the second is a standard example of a floating-point program called Spice. (Abstract)

Keywords—component, formatting, style, styling, insert (keywords)

I. INTRODUCTION (HEADING 1)

Pipeline structure in modern computer architecture is a common method used to increase the throughput of the system. One of the bottlenecks of the pipeline structure is the branch instruction. This is because, in a simple pipeline structure without branch prediction, each branch stalls the pipeline operation for a number of cycles, increasing the number of cycles required to execute the program. One way of improving the pipeline performance is to use a branch prediction algorithm. The main idea of branch prediction is to be able to predict the outcome of the branched instruction, whether it was taken or not taken. The system then speculatively executes the predicted outcome. In this way, when the actual outcome is realized, the system has already begun execution of the instruction and can continue without as many stalls. If the prediction is wrong, the system will continue as without prediction, requiring several stalls to execute the actual outcome. There are several ways to implement branch prediction. Three common branch prediction algorithms are Local prediction, Global prediction, and Tournament prediction, which is a combination of the Local and Global prediction algorithms.

In the Local prediction algorithm, the previously stored outcome of the current branch is used to predict if the same branch is taken or not taken. In Global prediction, the decision is made based on the history of the previous branches. As a more advanced algorithm for branch prediction, the Tournament predictor uses a selector state machine to determine which of the Local and Global predictions is more likely to give the correct prediction based on their previous accuracy, and chooses the prediction from that predictor.

As proof of concept, we have implemented the three aforementioned branch prediction algorithms using MATLAB, in order to compare the performance of each algorithm. To show the impact and effectiveness of each prediction scheme, the program was run for both integer and floating-point traces from Espresso and Spice, respectively.

II. METHODS

A. Selecting a Template (Heading 2)

To simulate the performance of the three prediction algorithms we have implemented each scheme in MATLAB

code. As input entries, we have used the Espresso and Spice benchmarks. Espresso is an example of an integer-based program and Spice is an example of a floating-point program. The procedure which we have used to find and predict the branches among the normal trace entries follows several steps which are described as follows:

- First, we need to check whether the new entry is in the Branch Target Buffer (BTB) or not as shown in Fig. 1.
- If the instruction is not in the BTB, we should check if this new entry is a branch or not. We have used the following code for this part.

```
if (trace_dec(i)+4 ~= trace_dec(i+1))
    Flag_trace_Jump = 1 ;
else
    Flag_trace_Jump = 0 ;
end
```

If it is not a branch, we consider it as a normal command. In that case we do not need to modify the BTB, and we can continue to the next instruction.

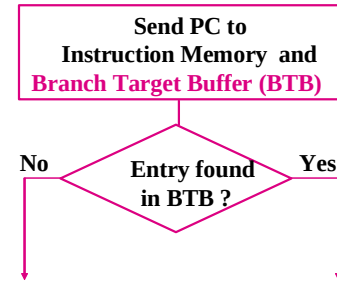


Fig. 1 Step 1 in the algorithm

- If the entry is not in the BTB but is a branch, we face a miss in the table. We add the counter for the miss numbers and modify the BTB entries for future predictions as shown in Fig. 2. If it is the first time we face this entry, the entry in the BTB needs to be initialized for the first time.

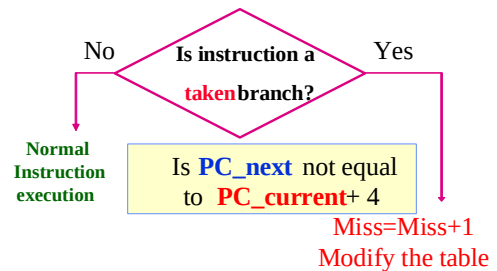


Fig. 2 Step 2.A in the algorithm

- The other condition in Fig. 1, is when the current PC is already in the BTB. In this case we have a hit, and we should increment the counter for number of hits by 1. Then we need to check if the target address for this entry is correct. If the target address is not correct, the wrong prediction counter should be incremented and the BTB modified accordingly. If the target address is correct, then the right counter should

be incremented. In this case, we should modify the BTB for the current entry based on Local, Global, and Selector state machines which are shown in Fig. 3 and 4.

As shown in Fig. 3 and Fig. 4, for every entry we have independent predictions from the Local and Global predictors. Each time, the outcome from one of these is selected as the final prediction based on the current status of selector. After the determination of final prediction, we need to modify the Local, Global and Selector states based on the actual branch outcome.

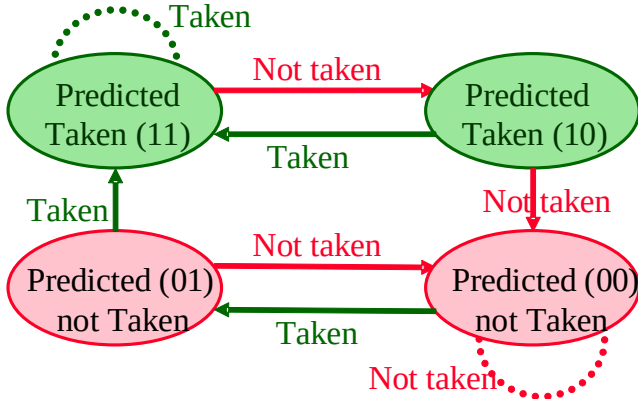


Fig. 3 State Machine for the Global and Local Predictor

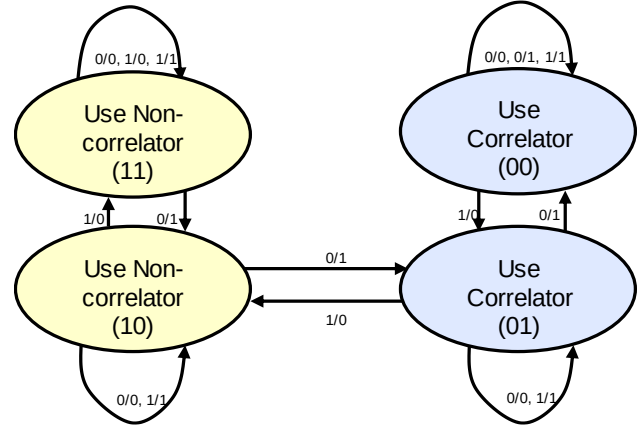


Fig. 4 State machine for the Tournament predictor

- If the current entry is in the BTB but there is no jump in the trace, it means we face a not taken branch. In that case, we need to modify the BTB based on the predictor's outcome while we know the branch is not taken.
- There is another condition in which we have taken branch and the BTB entry is already occupied by another entry. In this case, we consider it a collision between two different branches which points to the same location in the BTB. The collision counter is increased and the BTB updated accordingly.

Table 1 summary of the simulation results for all the benchmarks

Benchmarks	Spice FP	Doduc FP	Espresso Int	Li int
Entries	788262	724090	809371	716529
Number of Hits	120284	114845	129585	119603
Number of Misses	6597	5833	452	3137
Wrong Address	8221	3576	671	9953
Right Prediction Local	111874	111609	123294	107872
Right Prediction Global	113588	111755	123309	108671
Right Prediction Final	113533	111761	123316	108720
Wrong Prediction Local	16631	6812	6962	21684
Wrong Prediction Global	14917	6666	6947	20885
Wrong Prediction Final	14972	6660	6940	20836
Collisions	6016	5114	110	2535
Number of Times local Selected	101944	109353	129532	111489
Number of Times Global Selected	18340	5492	53	8114

B. Results

In this section, the simulation results of the implemented algorithm are shown. We have used all 4 different available benchmarks to achieve more accurate insight into the implemented algorithm. The summary of the simulation results is shown in Table 1.

As we can see in the table the summation of the Wrong predictions from Tournament predictor is the same as the number of Hits as expected. To gain a better understanding and clear view of the performance of the program, we have calculated the hit rates, accuracy of each predictor, miss predictions and CPI in Table 2. As we can see in Table 2 for all 4 benchmarks the tournament miss prediction rate is the best and is very close the global predictor. While the local predictor miss rate is higher compared to the global predictor as we expected. The Espresso Int benchmark

shows the best results in this implementation as it was shown in the lecture slides as well. The miss prediction rate in Doduc FP is 5.36% for global predictor, 5.37% for local predictor and 5.36% for the tournament predictor which are the least miss prediction rates among all the benchmarks. On the other hand the Li Int has 18.13% miss predictions for local predictor, 17.46% for the global predictor and 17.42% for the tournament predictor which are the worst miss prediction rates among all the benchmarks. Note that the provided Spice FP benchmark (even the last version) had some problems in the trace file and we had to remove those problematic entries. It is possible that part of the unexpected results for the Spice FP benchmark is due to this modification.

As we can see in Table 2 the worst miss prediction from the global predictor is LI int which is an integer-based program. Table 2 shows Espresso benchmark has lower miss prediction rates among all the other while we expect to have such results in a floating-point-based program. In Fig.5 the simulated CPI is shown for all the benchmarks with the predictors and without predictors. For both cases without predictors and with predictors, 1 stall for each branch is considered. As we can see the Li Int has the worst CPI, which means to execute the program the system requires more cycles to complete it. That is because it has more miss predictions and as a result more stalls.

C. Conclusion

As we have shown the prediction schemes improve the CPI for all benchmarks. For the Doduc FP and Espresso benchmark, we can see considerable improvement in the CPI and the performance of the system. Clearly for a pipeline CPU where we have more pipeline stages, we need to consider more stalls for each mispredicted branch resulting in a greater difference between the CPI with predictors and without predictors; which shows the importance of using branch predictors for deeply pipelined structures.

Table 2 Statistics for the simulated benchmarks

Benchmarks	Spice FP	Doduc FP	Espresso Int	Li int
Hit Rate	94.80	95.17	99.65	97.44
Accuracy of Tournament	94.39	97.31	95.16	90.90
Accuracy of Local	93.01	97.18	95.15	90.19
Accuracy of Global	94.43	97.31	95.16	90.86
Miss Prediction rate	12.45	5.80	5.36	17.42
Miss Prediction rate Local	13.83	5.93	5.37	18.13
Miss Prediction Rate Global	12.40	5.80	5.36	17.46
Prediction from Local Predictor	84.75	95.22	99.96	93.22
Prediction from Global Predictor	15.25	4.78	0.04	6.78
CPI	1.12	1.06	1.05	1.17

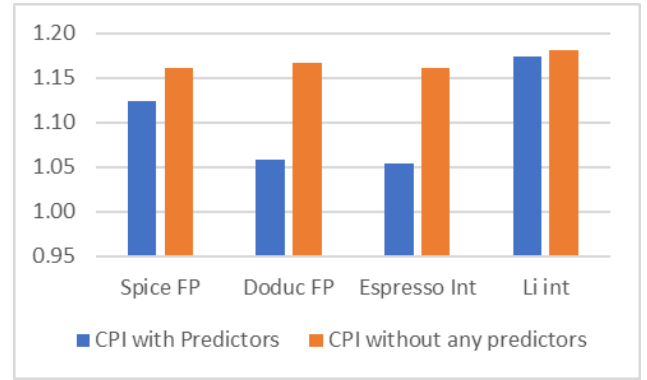


Fig. 5 the simulated CPI for different benchmarks